Multivariate Time-Varying Process Simulation

A Factor-Model-Based Generator for Static, Drifting, and Abrupt-Change Regimes

Language: Python
Application Domain: Multivariate Statistical Analysis / Change-Point Detection
Date: March 2026

1. Introduction

This document describes a Python script that generates synthetic multivariate time series data under three distinct statistical regimes: a stationary phase, a time-varying drift phase, and a time-varying phase with a superimposed abrupt mean shift. The primary purpose of the script is to produce benchmark datasets with fully known ground-truth properties for the development and evaluation of multivariate statistical analysis methods, particularly those concerned with change-point detection, regime identification, and non-stationarity testing.

The data generation model follows the standard linear factor model: a p-dimensional observation vector is expressed as a linear combination of a small number of latent common factors, plus additive Gaussian noise. This structure produces data with realistic cross-variable correlation while keeping the generative mechanism transparent and analytically tractable. By controlling the factor loadings matrix, the noise level, the drift coefficients, and the magnitude of the abrupt change, the script produces data whose statistical properties are fully specified and reproducible.

The script produces two output CSV files: `Y_f_off1dynamics.csv`, which contains the full three-regime dataset with known change points, and `Y_f_off1static.csv`, which contains a stationary baseline dataset of the same length. These files are designed to serve as matched pairs for benchmarking algorithms that distinguish between stationary and non-stationary multivariate processes.

2. Data Generation Model

2.1 Linear Factor Model

The underlying generative mechanism is the linear factor model, a standard framework in multivariate statistics for representing high-dimensional data as driven by a small number of latent common factors:

$$x_k = A \cdot t_k + e_k$$

Symbol	Dimension	Description
x_k	6 × 1	Observation vector at time step k; a 6-dimensional multivariate realisation
A	6 × 3	Factor loadings matrix (fixed); encodes the linear relationship between the 3 latent factors and the 6 observed variables
t_k	3 × 1	Latent factor vector at time step k; each component drawn independently from a zero-mean Gaussian distribution
e_k	6 × 1	Idiosyncratic noise vector; each component drawn i.i.d. from N(0, 0.04)

The factor model posits that the cross-variable correlation structure of the observed data is entirely explained by the shared latent factors t_k, while the noise term e_k represents variation specific to each observed variable. This is the standard assumption underlying principal component analysis (PCA) and related latent-variable methods.

2.2 Factor Loadings Matrix A

The loadings matrix A is fixed and shared across all three generator functions:

```
A = [[-0.2310, -0.0816, -0.2662],
     [-0.3241,  0.7055, -0.2158],
     [-0.2170, -0.3056, -0.5207],
     [-0.4089, -0.3442, -0.4501],
     [-0.6408,  0.3102,  0.2372],
     [-0.4655, -0.4330,  0.5938]]
```

Each column of A is the loading vector of one latent factor, describing how that factor projects onto the 6-dimensional observation space. Each row describes the sensitivity of one observed variable to all three latent factors. The columns are not required to be orthonormal, though in PCA applications the loadings are typically derived from the eigenvectors of the sample covariance matrix.

2.3 Latent Factor Distributions

At each time step k, the three latent factors are drawn independently from zero-mean Gaussian distributions with differing standard deviations, encoding a variance hierarchy across the three underlying dimensions of variation:

Factor	Distribution	Standard Deviation	Interpretation
t_1	$N(0, 1^2)$	1.0	Dominant mode of variation; largest contribution to

Factor	Distribution	Standard Deviation	Interpretation
			total variance
t_2	N(0, 0.8²)	0.8	Secondary mode; intermediate variance contribution
t_3	N(0, 0.6²)	0.6	Tertiary mode; smallest variance contribution

The decreasing variance sequence (1.0, 0.8, 0.6) mirrors the eigenvalue ordering of a PCA decomposition, where the first principal component accounts for the largest fraction of total variance. Successive factors explain progressively less variance, consistent with the typical structure of real multivariate datasets in which a small number of latent dimensions drive most of the observed variation.

3. Generator Functions

Three generator functions are defined, each returning a single 6 × 1 observation vector `xk` for one time step. All three share the same factor model core and differ only in what additive term, if any, is appended to the base output.

3.1 static() — Stationary Phase

The `static()` function generates a single observation from the pure factor model with no time-index argument. It represents the stationary, time-homogeneous regime in which the data-generating distribution does not change over time.

```
def static():
    A  = <6×3 loadings matrix>
    t1 = np.random.normal(0, 1.0)     # Latent factor 1
    t2 = np.random.normal(0, 0.8)     # Latent factor 2
    t3 = np.random.normal(0, 0.6)     # Latent factor 3
    t  = np.array([[t1, t2, t3]]).T  # Shape: (3, 1)
    e  = np.random.normal(0, 0.2, [6, 1])  # Noise: N(0, 0.04)
    xk = np.dot(A, t) + e
    return xk                          # Shape: (6, 1)
```

The distribution of `xk` under this function is multivariate Gaussian: $x_k \sim N(0, A\Sigma_t A^T + 0.04\, I_6)$, where Σ_t = diag(1, 0.64, 0.36). Successive calls are statistically independent — there is no temporal correlation between time steps.

3.2 time_Varying(i) — Drifting Phase

The `time_Varying(i)` function adds a deterministic, linearly time-varying mean vector to the base factor model output. The drift is anchored at sample index i = 300, so the drift contribution is identically zero at the phase transition and grows linearly in magnitude thereafter.

```
def time_Varying(i):
    # ... same A, t, e generation as static() ...
    timevary = np.array([[
         0.005*(i-300),   # Variable 1: positive linear drift
         0,               # Variable 2: no drift
        -0.004*(i-300),   # Variable 3: negative linear drift
         0,               # Variable 4: no drift
         0,               # Variable 5: no drift
         0.007*(i-300)    # Variable 6: positive drift (steepest slope)
    ]]).T
    xk = np.dot(A, t) + e + timevary
    return xk
```

The time-varying drift vector models a gradual, continuous shift in the mean of the observed process — a common feature of non-stationary time series arising from slowly evolving latent conditions. The linear form with anchor at i = 300 ensures distributional continuity at the phase boundary: the mean vector is zero at i = 300 and diverges linearly in subsequent steps.

Variable Index	Drift Rate	Cumulative Drift at i = 800	Direction
1	0.005 per step	0.005 × 500 = +2.5	Positive
2	0 (no drift)	0	—
3	−0.004 per step	−0.004 × 500 = −2.0	Negative
4	0 (no drift)	0	—
5	0 (no drift)	0	—
6	0.007 per step	0.007 × 500 = +3.5	Positive (steepest)

The drift is selective — only variables 1, 3, and 6 are affected — while variables 2, 4, and 5 remain stationary throughout this phase. This sparse drift structure tests whether a change-detection algorithm can localise which variables have undergone a distributional shift.

3.3 time_Varying_fault(i) — Drifting Phase with Abrupt Mean Shift

The `time_Varying_fault(i)` function generates the same time-varying drift output as `time_Varying(i)` and then superimposes a constant additive mean shift on all six variables simultaneously. This represents an abrupt, step-like change in the process mean occurring on top of the pre-existing drift.

```
def time_Varying_fault(i):
    # ... same base output as time_Varying(i) ...
    fault = np.array([[1, 1, 1, 1, 1, 1]]).T  # Direction: uniform shift
    xk_fault = xk + 6 * fault                  # Magnitude: 6 units
    return xk_fault
```

The abrupt shift is modelled as a scalar multiple of the all-ones vector, producing a uniform positive

offset of 6 units across all six variables. The combined generative model for this phase is:

$$x_k = A \cdot t_k + e_k + f(i) + 6 \cdot 1_6$$

where f(i) denotes the time-varying drift vector. The shift magnitude of 6 units is large relative to the noise standard deviation (σ = 0.2), corresponding to a signal-to-noise ratio of 30. This ensures the step change is clearly detectable in the mean, providing a strong test case for change-point detection algorithms.

Note on terminology: The Python variable and function name uses the word 'fault' (a term from engineering diagnostics). In the present statistical context, this term simply denotes an abrupt, step-like change in the mean of the data-generating distribution — a standard concept in change-point analysis and structural break testing.

4. Simulation Pipeline

4.1 Three-Regime Dynamic Dataset (Y_f_off1dynamics.csv)

The first simulation loop generates n = 1000 time steps partitioned across three consecutive regimes by selecting the appropriate generator function based on the sample index i:

```
n = 1000
xk_final = np.array([np.zeros(6)]).T   # (6,1) placeholder

for i in range(1, n+1):
    if i <= 300:                        # Regime 1: Stationary  (steps 1-300)
        xk = static()
    elif i > 300 and i <= 800:          # Regime 2: Drifting    (steps 301-800)
        xk = time_Varying(i)
    else:                               # Regime 3: Drift + abrupt shift (801-1000)
        xk = time_Varying_fault(i)
    xk_final = np.c_[xk_final, xk]

xk_final = np.delete(xk_final, 0, axis=1).T   # Shape: (1000, 6)
np.savetxt('Y_f_off1dynamics.csv', xk_final, delimiter=',')
```

Regime	Step Range	Generator	Statistical Character	Duration
1	1 – 300	`static()`	Stationary; constant mean μ = 0 and constant covariance Σ	300 steps (30%)
2	301 – 800	`time_Varying(i)`	Non-stationary; linearly drifting mean; constant covariance	500 steps (50%)
3	801 – 1000	`time_Varying_fault(i)`	Non-stationary; drift continues; abrupt mean	200 steps (20%)

Regime	Step Range	Generator	Statistical Character	Duration
			shift of +6 at i = 801	

Change-point structure: The dataset contains two known change points: a smooth onset of drift at i = 301 (a gradual, continuous non-stationarity) and a step change at i = 801 (an abrupt distributional shift). The ground-truth knowledge of these change points enables precise evaluation of detection delay, false alarm rate, and localisation accuracy for any algorithm applied to this dataset.

4.2 Stationary Baseline Dataset (Y_f_off1static.csv)

The second simulation loop generates a complementary stationary dataset of the same length (n = 1000) by calling `static()` at every time step. This dataset represents a purely time-homogeneous process with no drift or structural breaks, and serves as the null-hypothesis reference distribution for comparison against the dynamic dataset.

```
n = 1000
xk_final_off = np.array([np.zeros(6)]).T

for i in range(1, n+1):
    xk_off = static()
    xk_final_off = np.c_[xk_final_off, xk_off]

xk_final_off = np.delete(xk_final_off, 0, axis=1).T
np.savetxt('Y_f_off1static.csv', xk_final_off, delimiter=',')
```

In a typical analysis workflow, the stationary dataset is used to estimate model parameters (e.g., covariance matrix, principal components, control limits) under the in-control or null hypothesis. The dynamic dataset is then used as the test set, where the analyst evaluates whether the chosen method correctly identifies the two change points.

4.3 Array Construction Pattern

Both simulation loops use an identical incremental column-append pattern. A zero-column placeholder is created before the loop, and `np.c_[]` appends each new 6 × 1 vector as a column at each iteration. After the loop, the placeholder column is removed via `np.delete(..., 0, axis=1)`, and the matrix is transposed to the standard *(samples × variables)* format.

Step	Operation	Resulting Shape	Purpose
1	`np.zeros(6) → reshape to (6,1)`	(6, 1)	Initialise placeholder for column appending
2	`np.c_[current, xk] (per step)`	(6, k+1)	Append new observation as a column
3	`np.delete(..., 0,`	(6, 1000)	Remove the initial zero-column

Step	Operation	Resulting Shape	Purpose
	`axis=1)`		placeholder
4	`.T`	(1000, 6)	Transpose to standard data matrix (rows = time steps)
5	`np.savetxt(..., delimiter=',')`	—	Write to CSV with comma delimiter, no header

5. Output Files

Filename	Shape	Content	Role in Analysis
`Y_f_off1dynamics.csv`	1000 × 6	Three-regime dataset: 300 stationary + 500 drifting + 200 drifting-with-shift steps	Test dataset; contains two ground-truth change points at i = 301 and i = 801
`Y_f_off1static.csv`	1000 × 6	1000 purely stationary samples drawn i.i.d. from $N(0, A\Sigma_t A^T + 0.04I_6)$	Baseline / reference dataset; represents the null hypothesis of stationarity

Both files are written in standard CSV format with comma delimiters, no header row, and 64-bit floating-point precision. Each row is one time step and each column is one observed variable. The file naming convention implies membership in a larger benchmark family (the suffix '1' suggesting the first in a series of scenarios with varying drift rates, shift magnitudes, or loadings matrices).

6. Recommended Extensions

Extension	Description
Reproducibility	`np.random.seed(seed)` at script start to ensure identical output across runs.
Sparse shift direction	Replace the all-ones shift vector with a sparse unit vector (e.g., shifting only one or two variables) to test change localisation, not just global detection.
Parameterised scenarios	Wrap the two simulation loops in a function that accepts drift rates, shift magnitude, and n as arguments, enabling systematic scenario sweeps for sensitivity analysis.
Autocorrelated factors	Replace the i.i.d. Gaussian factor draws with a VAR(1) process to introduce temporal autocorrelation, producing more realistic

Extension	Description
	multivariate time series.
Pre-allocated arrays	Replace the column-append loop with `xk_final = np.zeros((6, n))` filled by column index for O(n) memory complexity.
Metadata output	Write a companion JSON or header file recording the change-point indices, drift coefficients, shift magnitude, and random seed to facilitate automated benchmark evaluation.

7. Summary

This script provides a concise and self-contained data generator for multivariate time-varying processes. Three generator functions implement distinct statistical regimes — stationary, linearly drifting, and drifting with an abrupt mean shift — all built on the same linear factor model with a fixed loadings matrix A and i.i.d. Gaussian noise. The known ground-truth structure of the generated data makes it directly suitable for benchmarking change-point detection, non-stationarity testing, and regime identification algorithms.

Two output files are produced: `Y_f_off1dynamics.csv` (1000 × 6, three-regime) contains two ground-truth change points at i = 301 and i = 801, while `Y_f_off1static.csv` (1000 × 6, fully stationary) provides a matched null-hypothesis reference. Together, they form a paired benchmark dataset suitable for rigorous algorithm evaluation.

The primary limitations of the current implementation are the assumption of temporal independence between steps, the linear drift model, and the absence of a fixed random seed for reproducibility. These can be addressed through the extensions described in Section 8, enabling a broader and more representative benchmark library.

References

[1] Basseville, M., & Nikiforov, I. V. (1993). *Detection of Abrupt Changes: Theory and Application*. Prentice Hall.

[2] Stock, J. H., & Watson, M. W. (2002). Forecasting using principal components from a large number of predictors. *Journal of the American Statistical Association*, 97(460), 1167–1179.

[3] Bai, J., & Perron, P. (1998). Estimating and testing linear models with multiple structural changes. *Econometrica*, 66(1), 47–78.

[4] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.